

Theory of Concurrency

Piotr Hofman

Theoretical aspects of concurrent systems

- Email: piotr.hofman@uw.edu.pl
- Office: CeNT 3.109
- Tutorials: Vaios Michalakis

Outline

- ① How do we model concurrent systems?
 - labelled transition systems and Kripke structures,
 - process algebras,
 - Petri nets.
- ② How do we specify and verify their behaviour?
 - traces and languages of infinite words,
 - Büchi automata,
 - temporal logics: LTL and CTL.
- ③ When are two systems behaviourally equivalent?
 - simulation and bisimulation.

Assessment methods and assessment criteria

Oral exam

- At the end of the semester I will provide a list of questions that may appear on the exam.
- The oral exam consists of three randomly selected questions.
- Each question is graded from 0 to 5 points.

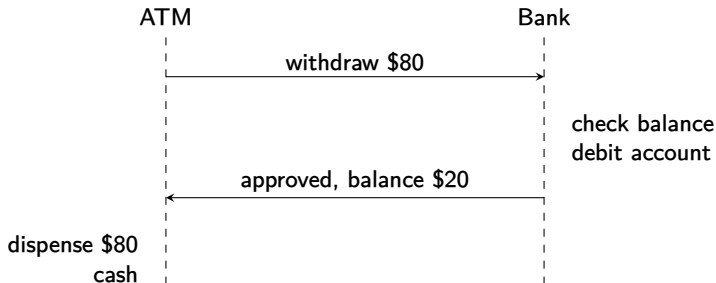
Points	Grade
$[0, 8]$	2
$(8, 10]$	3
$(10, 11.5]$	3+
$(11.5, 13]$	4
$(13, 14]$	4+
$(14, 15]$	5

Let us start

One execution: an ATM withdrawal

Message exchange

The ATM sends a withdrawal request to the bank. The bank processes it and sends a reply.



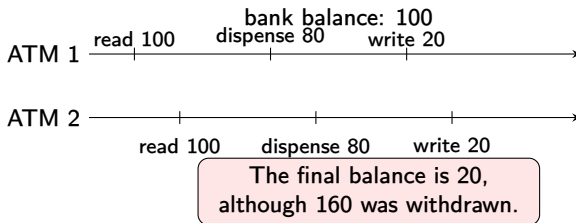
One possible execution

Request → bank processing → reply → cash dispensed.

Three basic threats in parallel programming

Race condition / inconsistent shared state

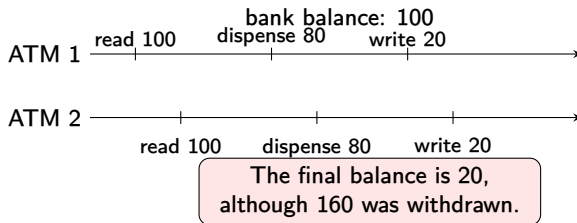
Two ATMs access the same account concurrently.



Three basic threats in parallel programming

Race condition / inconsistent shared state

Two ATMs access the same account concurrently.



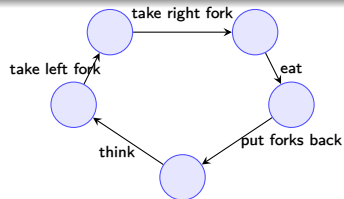
Lesson

Correctness may depend on all possible interleavings of concurrent actions.

Three basic threats in parallel programming

Deadlock

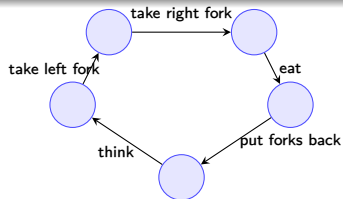
Consider philosophers following the protocol below.



Three basic threats in parallel programming

Deadlock

Consider philosophers following the protocol below.



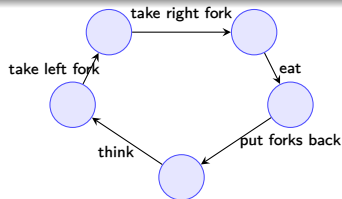
Deadlock scenario

If all philosophers first take their left fork, then every philosopher waits for the right fork and no one can proceed.

Three basic threats in parallel programming

Deadlock

Consider philosophers following the protocol below.



Deadlock scenario

If all philosophers first take their left fork, then every philosopher waits for the right fork and no one can proceed.

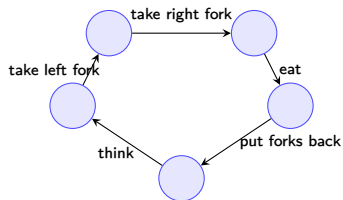
Typical solution

Break circular waiting, e.g. acquire forks according to a fixed global order.

Three basic threats in parallel programming

Starvation

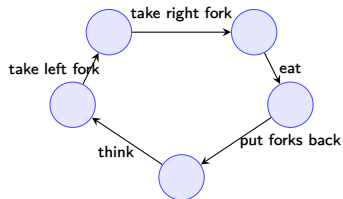
A system may keep making progress while one process is repeatedly prevented from making progress.



Three basic threats in parallel programming

Starvation

A system may keep making progress while one process is repeatedly prevented from making progress.



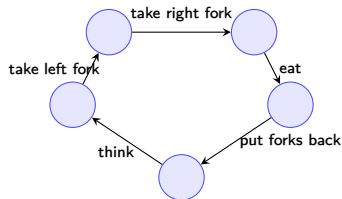
Typical solution

Use a fair scheduling or allocation policy, e.g. FIFO access or bounded waiting.

Three basic threats in parallel programming

Starvation

A system may keep making progress while one process is repeatedly prevented from making progress.



Typical solution

Use a fair scheduling or allocation policy, e.g. FIFO access or bounded waiting.

Contrast with deadlock

In a deadlock nobody progresses. Under starvation, the system progresses, but not fairly.

Goal of formal modelling

What we want

We want mathematical models that describe all possible executions of a concurrent system and allow us to state which executions are correct.

- Race condition: “something bad never happens.”
- Deadlock freedom: “the system can always make progress.”
- Starvation freedom: “a process that keeps requesting access eventually succeeds.”

Goal of formal modelling

What we want

We want mathematical models that describe all possible executions of a concurrent system and allow us to state which executions are correct.

- Race condition: “something bad never happens.”
- Deadlock freedom: “the system can always make progress.”
- Starvation freedom: “a process that keeps requesting access eventually succeeds.”

Key difficulty

The behaviour of a concurrent system is not one execution, but a set of possible executions.

How can we observe a running system?

For now, we consider two simple forms of observation.

Logs

A running system records selected events as a log.

Debugger

A debugger allows us to inspect the system state step by step.

Limitations

- Recording every event or every variable may be prohibitively expensive.
- Observing the system may itself change its behaviour, e.g. by affecting execution timing.

Model

Model

A model is a simplified representation of a system that preserves the aspects relevant to the question being studied.

Model

Model

A model is a simplified representation of a system that preserves the aspects relevant to the question being studied.

Abstraction

We deliberately forget some information.

- Logs expose only selected actions; other actions may be silent.
- State-based models expose only selected variables or predicates.

Model

Model

A model is a simplified representation of a system that preserves the aspects relevant to the question being studied.

Abstraction

We deliberately forget some information.

- Logs expose only selected actions; other actions may be silent.
- State-based models expose only selected variables or predicates.

Consequence

A proof about the model is meaningful only under the assumption that the model captures the relevant behaviour of the real system.

Kripke structures and labelled transition systems

General concept

- A system is always in some state, also called a configuration.
- The state may evolve during execution.
- States are vertices; possible evolutions are edges.

Kripke structures

States are labelled by propositions that are true in the current state.

Labelled transition systems

Transitions are labelled by actions performed by the system.

Labelled transition systems

Definition

A labelled transition system is a tuple

$$T = (S, I, \Sigma, \rightarrow)$$

where:

- 1 S is a set of states, possibly infinite;
- 2 $I \subseteq S$ is a set of initial states;
- 3 Σ is a set of actions;
- 4 $\rightarrow \subseteq S \times \Sigma \times S$ is a transition relation.

We write $s \xrightarrow{a} s'$ for $(s, a, s') \in \rightarrow$.

Labelled transition systems

Definition

A labelled transition system is a tuple

$$T = (S, I, \Sigma, \rightarrow)$$

where:

- 1 S is a set of states, possibly infinite;
- 2 $I \subseteq S$ is a set of initial states;
- 3 Σ is a set of actions;
- 4 $\rightarrow \subseteq S \times \Sigma \times S$ is a transition relation.

We write $s \xrightarrow{a} s'$ for $(s, a, s') \in \rightarrow$.

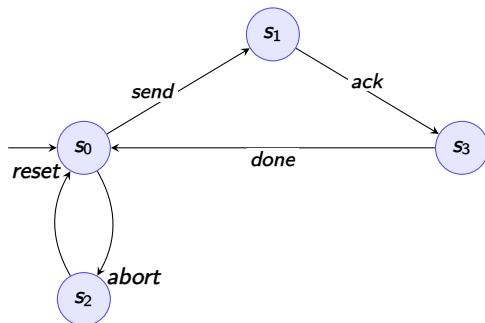
Runs and traces

An infinite run is

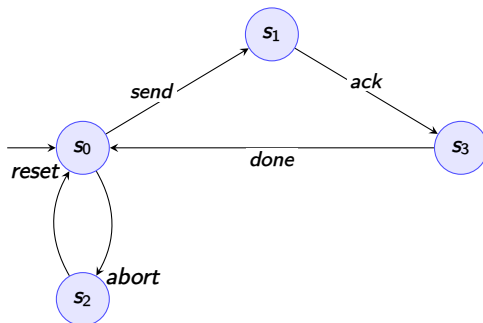
$$s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} \dots$$

where $s_0 \in I$ and $s_i \xrightarrow{a_i} s_{i+1}$ for all $i \geq 0$. Its trace is the infinite word $a_0 a_1 a_2 \dots \in \Sigma^\omega$.

Example: simple communication protocol



Example: simple communication protocol



- Initial state: s_0 .
- Actions: $\Sigma = \{\text{send}, \text{ack}, \text{abort}, \text{reset}, \text{done}\}$.
- Example infinite traces:

$(\text{send ack done})^\omega$

and

$\text{abort reset } (\text{send ack done})^\omega$.

Kripke structures

Let $\mathcal{AP}$ be a set of atomic propositions.

Kripke structures

Let $\mathbb{AP}$ be a set of atomic propositions.

Definition

A Kripke structure over $\mathbb{AP}$ is a tuple

$$M = (S, I, R, L)$$

where:

- 1 S is a finite set of states;
- 2 $I \subseteq S$ is a set of initial states;
- 3 $R \subseteq S \times S$ is a left-total transition relation:

$$\forall s \in S \exists s' \in S \quad (s, s') \in R;$$

- 4 $L : S \rightarrow 2^{\mathbb{AP}}$ labels each state by the propositions true in that state.

Runs and traces of Kripke structures

Run

An infinite run of a Kripke structure $M = (S, I, R, L)$ is a sequence

$$s_0 s_1 s_2 \dots$$

such that $s_0 \in I$ and $(s_i, s_{i+1}) \in R$ for every $i \geq 0$.

Runs and traces of Kripke structures

Run

An infinite run of a Kripke structure $M = (S, I, R, L)$ is a sequence

$$s_0 s_1 s_2 \dots$$

such that $s_0 \in I$ and $(s_i, s_{i+1}) \in R$ for every $i \geq 0$.

Trace

The trace of this run is

$$L(s_0)L(s_1)L(s_2)\dots \in (2^{\mathbb{AP}})^\omega.$$

The set of traces of all infinite runs of M is denoted by $\text{TR}(M)$.

Runs and traces of Kripke structures

Run

An infinite run of a Kripke structure $M = (S, I, R, L)$ is a sequence

$$s_0 s_1 s_2 \cdots$$

such that $s_0 \in I$ and $(s_i, s_{i+1}) \in R$ for every $i \geq 0$.

Trace

The trace of this run is

$$L(s_0)L(s_1)L(s_2)\cdots \in (2^{\mathbb{AP}})^\omega.$$

The set of traces of all infinite runs of M is denoted by $\text{TR}(M)$.

Comparison

LTS traces record actions. Kripke traces record properties of states.

Pros and cons of LTS and Kripke structures

Advantages

- They are simple and intuitive.
- They provide a precise operational model.
- They are useful for defining semantics of richer modelling formalisms.

Pros and cons of LTS and Kripke structures

Advantages

- They are simple and intuitive.
- They provide a precise operational model.
- They are useful for defining semantics of richer modelling formalisms.

Limitations

- Explicit models may be very large.
- Large transition systems are difficult to inspect manually.
- They often hide the structural or modular description from which the state space was generated.

Pros and cons of LTS and Kripke structures

Advantages

- They are simple and intuitive.
- They provide a precise operational model.
- They are useful for defining semantics of richer modelling formalisms.

Limitations

- Explicit models may be very large.
- Large transition systems are difficult to inspect manually.
- They often hide the structural or modular description from which the state space was generated.

For the first few lectures we will use them as basic models of systems.

The language perspective

Idea

A system model defines a language of possible behaviours. A specification can also be viewed as a language.

The language perspective

Idea

A system model defines a language of possible behaviours. A specification can also be viewed as a language.

Error language

Let E be the set of all erroneous behaviours. Then

$$\text{TR}(M) \cap E = \emptyset$$

means that no behaviour of the system is erroneous.

The language perspective

Idea

A system model defines a language of possible behaviours. A specification can also be viewed as a language.

Error language

Let E be the set of all erroneous behaviours. Then

$$\text{TR}(M) \cap E = \emptyset$$

means that no behaviour of the system is erroneous.

Correct-behaviour language

Let P be the set of all acceptable behaviours. Then

$$\text{TR}(M) \subseteq P$$

means that every behaviour of the system satisfies the specification.

Exercise: five philosophers

Modelling task

- Choose atomic propositions sufficient to describe the relevant behaviour.
- Describe the states and transitions of a corresponding Kripke structure.
- Draw a representative fragment of this Kripke structure.

Exercise: five philosophers

Modelling task

- Choose atomic propositions sufficient to describe the relevant behaviour.
- Describe the states and transitions of a corresponding Kripke structure.
- Draw a representative fragment of this Kripke structure.

Properties to specify

- A philosopher can eat only while holding both forks.
- Every philosopher eats infinitely often.
- Rene Descartes eats infinitely often and thinks infinitely often.

Exercise: five philosophers

Modelling task

- Choose atomic propositions sufficient to describe the relevant behaviour.
- Describe the states and transitions of a corresponding Kripke structure.
- Draw a representative fragment of this Kripke structure.

Properties to specify

- A philosopher can eat only while holding both forks.
- Every philosopher eats infinitely often.
- Rene Descartes eats infinitely often and thinks infinitely often.

Question

How can these properties be specified over infinite traces?

Why infinite-word automata?

Reactive systems

Concurrent and reactive systems may run forever. Their complete behaviours are naturally represented by infinite words.

Why infinite-word automata?

Reactive systems

Concurrent and reactive systems may run forever. Their complete behaviours are naturally represented by infinite words.

Finite automata are not enough

Ordinary finite automata recognize languages of finite words. We need automata whose acceptance condition is defined for infinite runs.

Why infinite-word automata?

Reactive systems

Concurrent and reactive systems may run forever. Their complete behaviours are naturally represented by infinite words.

Finite automata are not enough

Ordinary finite automata recognize languages of finite words. We need automata whose acceptance condition is defined for infinite runs.

Main question

How should an automaton accept an infinite word?

Finite automata

Let Σ be a finite alphabet.

Definition

A nondeterministic finite automaton is a tuple

$$A = (Q, \Sigma, I, F, \delta)$$

where:

- 1 Q is a finite set of states;
- 2 $I \subseteq Q$ is a set of initial states;
- 3 $F \subseteq Q$ is a set of accepting states;
- 4 $\delta \subseteq Q \times \Sigma \times Q$ is a transition relation.

Finite automata

Let Σ be a finite alphabet.

Definition

A nondeterministic finite automaton is a tuple

$$A = (Q, \Sigma, I, F, \delta)$$

where:

- 1 Q is a finite set of states;
- 2 $I \subseteq Q$ is a set of initial states;
- 3 $F \subseteq Q$ is a set of accepting states;
- 4 $\delta \subseteq Q \times \Sigma \times Q$ is a transition relation.

Acceptance

A finite word is accepted if it labels a path from an initial state to an accepting state.

Finite automata

Let Σ be a finite alphabet.

Definition

A nondeterministic finite automaton is a tuple

$$A = (Q, \Sigma, I, F, \delta)$$

where:

- 1 Q is a finite set of states;
- 2 $I \subseteq Q$ is a set of initial states;
- 3 $F \subseteq Q$ is a set of accepting states;
- 4 $\delta \subseteq Q \times \Sigma \times Q$ is a transition relation.

Acceptance

A finite word is accepted if it labels a path from an initial state to an accepting state.

Problem

Our traces are infinite words.

Acceptance conditions for infinite words

Let

$$q_0 \xrightarrow{a_0} q_1 \xrightarrow{a_1} q_2 \xrightarrow{a_2} \dots$$

be an infinite run.

Acceptance conditions for infinite words

Let

$$q_0 \xrightarrow{a_0} q_1 \xrightarrow{a_1} q_2 \xrightarrow{a_2} \dots$$

be an infinite run.

- Attempt 1: acceptance depends on some finite prefix.

Acceptance conditions for infinite words

Let

$$q_0 \xrightarrow{a_0} q_1 \xrightarrow{a_1} q_2 \xrightarrow{a_2} \dots$$

be an infinite run.

- Attempt 1: acceptance depends on some finite prefix.
- Attempt 2: from some point on, the run stays only in accepting states.

Acceptance conditions for infinite words

Let

$$q_0 \xrightarrow{a_0} q_1 \xrightarrow{a_1} q_2 \xrightarrow{a_2} \dots$$

be an infinite run.

- Attempt 1: acceptance depends on some finite prefix.
- Attempt 2: from some point on, the run stays only in accepting states.
- Attempt 3: the run visits accepting states infinitely often.

Büchi idea

Acceptance should express behaviour that occurs infinitely often.

Acceptance conditions for infinite words

Let

$$q_0 \xrightarrow{a_0} q_1 \xrightarrow{a_1} q_2 \xrightarrow{a_2} \dots$$

be an infinite run.

- Attempt 1: acceptance depends on some finite prefix.
- Attempt 2: from some point on, the run stays only in accepting states.
- Attempt 3: the run visits accepting states infinitely often.
- Attempt 4: the run visits each of several accepting sets infinitely often.

Büchi idea

Acceptance should express behaviour that occurs infinitely often.

Büchi automata

Definition

A Büchi automaton is a tuple

$$A = (Q, \Sigma, I, F, \delta)$$

where Q, Σ, I, δ are as for a nondeterministic finite automaton, and $F \subseteq Q$ is the set of accepting states.

Büchi automata

Definition

A Büchi automaton is a tuple

$$A = (Q, \Sigma, I, F, \delta)$$

where Q, Σ, I, δ are as for a nondeterministic finite automaton, and $F \subseteq Q$ is the set of accepting states.

Acceptance

An infinite run

$$q_0 \xrightarrow{a_0} q_1 \xrightarrow{a_1} q_2 \xrightarrow{a_2} \dots$$

is accepting if it visits F infinitely often:

$$\{i \geq 0 \mid q_i \in F\} \text{ is infinite.}$$

Büchi automata

Definition

A Büchi automaton is a tuple

$$A = (Q, \Sigma, I, F, \delta)$$

where Q, Σ, I, δ are as for a nondeterministic finite automaton, and $F \subseteq Q$ is the set of accepting states.

Acceptance

An infinite run

$$q_0 \xrightarrow{a_0} q_1 \xrightarrow{a_1} q_2 \xrightarrow{a_2} \dots$$

is accepting if it visits F infinitely often:

$$\{i \geq 0 \mid q_i \in F\} \text{ is infinite.}$$

The language $\mathbb{L}(A)$ is the set of infinite words that have an accepting run in A .

Generalised Büchi automata

Definition

A generalised Büchi automaton has several accepting sets

$$\mathcal{F} = \{F_1, \dots, F_k\}, \quad F_j \subseteq Q.$$

Generalised Büchi automata

Definition

A generalised Büchi automaton has several accepting sets

$$\mathcal{F} = \{F_1, \dots, F_k\}, \quad F_j \subseteq Q.$$

Acceptance

An infinite run is accepting if it visits every accepting set infinitely often:

$$\forall j \in \{1, \dots, k\} \quad \{i \geq 0 \mid q_i \in F_j\} \text{ is infinite.}$$

Generalised Büchi automata

Definition

A generalised Büchi automaton has several accepting sets

$$\mathcal{F} = \{F_1, \dots, F_k\}, \quad F_j \subseteq Q.$$

Acceptance

An infinite run is accepting if it visits every accepting set infinitely often:

$$\forall j \in \{1, \dots, k\} \quad \{i \geq 0 \mid q_i \in F_j\} \text{ is infinite.}$$

Example intuition

This is useful for conjunctions of liveness requirements: each required event must happen infinitely often.

Büchi languages

Lemma

For every finite Kripke structure M there is a Büchi automaton A such that

$$L(A) = \text{TR}(M).$$

Büchi languages

Lemma

For every finite Kripke structure M there is a Büchi automaton A such that

$$L(A) = \text{TR}(M).$$

Lemma

Büchi and generalised Büchi automata recognize the same class of languages.

Büchi languages

Lemma

For every finite Kripke structure M there is a Büchi automaton A such that

$$L(A) = TR(M).$$

Lemma

Büchi and generalised Büchi automata recognize the same class of languages.

ω -regular languages

Languages recognized by Büchi automata are called ω -regular languages. They are closed under union, intersection, and complement.

Büchi languages

Nonemptiness of a Büchi automaton

A Büchi automaton is nonempty iff it has a reachable accepting cycle.
a finite prefix reaches the cycle, and then the cycle is repeated forever.

Büchi languages

Nonemptiness of a Büchi automaton

A Büchi automaton is nonempty iff it has a reachable accepting cycle.
a finite prefix reaches the cycle, and then the cycle is repeated forever.

Determinisation warning

Not every nondeterministic Büchi automaton has an equivalent deterministic Büchi automaton. Determinisation requires stronger acceptance conditions, e.g. parity acceptance.